



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Dia, Fecha:	Sabado, 27/07/2024
Hora de inicio:	08:50

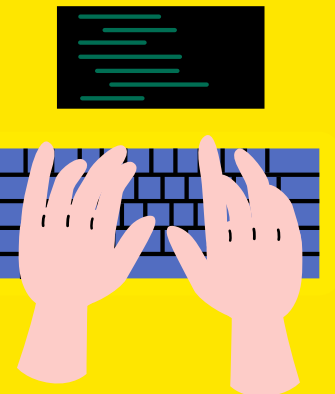
Laboratorio de Programacion de computadoras I, Sección A

Carlos Eduardo Soto Marroquín

Clase 1

PRÁCTICA DE PROGRAMACIÓN DE COMPUTADORAS

SECCIÓN A
CÓDIGO DEL CURSO 090



AGENDA DE LA CLASE:



Presentación del programa del curso

Distribución de la zona

Formulario de asistencia

Conceptos de programación

Foro de la semana

¿Dudas?



FORMATOS DE ENTREGA

TAREA: [PDC1]T#_#CARNET

PROYECTO: [PDC1]P#_#CARNET

PRACTICA: [PDC1]PRAC#_#CARNET



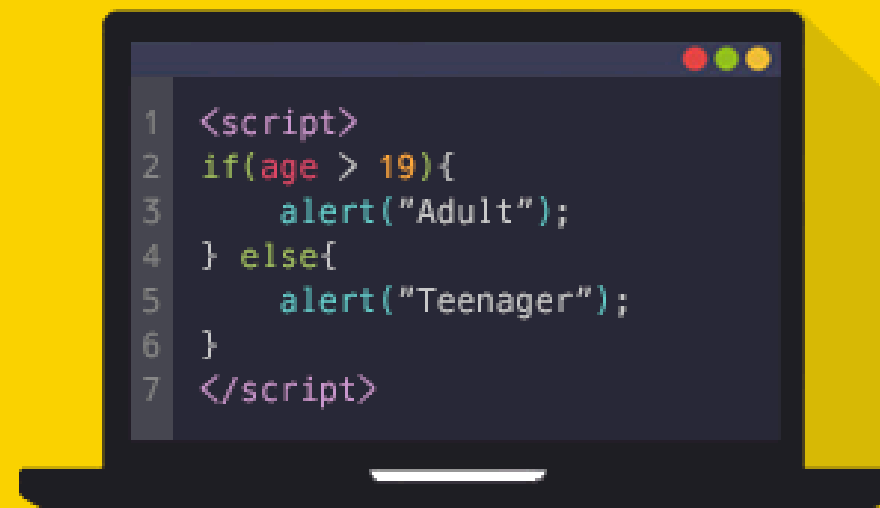
[HTTPS://WWW.MICROSOFT.COM/EN-US/EDUCATION/PRODUCTS/OFFICE](https://www.microsoft.com/en-us/education/products/office)

[illegible]

Introducción básica a la programación



JavaScript



Temas a tratar

Puntos clave:

- Breve Historia
- Sintaxis Básica
- Operadores
- Estructuras de control
- Funciones Básicas



Breve historia

JavaScript se introdujo en 1995 como una forma de agregar programas a páginas web en el navegador Netscape Navigator. El creador fue Brendan Eich bajo el nombre de "Mocha"



¿Qué es JS?

**Lenguaje de
programación
interpretado**



**Basado en prototipos
e imperativo**



**Es débilmente tipado
y dinámico**



Es utilizado principalmente en el lado del cliente, implementado como parte de un navegador web permitiendo crear la interacción con el usuario y páginas web dinámicas

Evolución de JavaScript

Historia de JavaScript

Versión de EcmaScript	Año
1	Junio 1997
2	Junio 1998
3	Diciembre 1999
4	Abandonada
5	Diciembre 2009
6	Junio 2015
7	En proceso

Sintaxis Básica

PDC1

La sintaxis de JS es similar a otros lenguajes de programación como Java y C. Las normas básicas que definen la sintaxis de JS son las siguientes:

- **No se toman en cuenta los espacios en blanco y las nuevas líneas.**
- **Es case-sensitive.**
- **No es necesario definir el tipo de variable.**
- **No es necesario terminar cada sentencia con el carácter de punto y coma.**
- **Se puede incluir comentarios.**

No se toman en cuenta los espacios en blanco y las nuevas líneas.

The diagram illustrates how JavaScript ignores whitespace and line breaks. It shows a code snippet with three annotations: 'espacios' (spaces) pointing to spaces in the code, 'saltos de línea' (line breaks) pointing to line breaks, and 'tabulacion' (indentation) pointing to the indentation of the object literal. The code is as follows:

```
1 let language = 'JavaScript'
2 let company = {
3   name: 'EDteam',
4   slogan: 'Nunca te detengas',
5   founded: 2015
6 }
```

The annotations are:

- espacios**: Points to the spaces between 'let' and 'language', and between '=' and ''JavaScript'.
- saltos de línea**: Points to the line breaks at the end of lines 1, 2, 3, 4, and 5.
- tabulacion**: Points to the indentation of the object literal on lines 2 through 6.

Es case-sensitive

```
//JS ES Case Sensitive, esto quiere decir que  
//var variable1 es diferente de var Variable1  
var variable1 = 1;  
var Variable = 1;  
// variable1 ≠ Variable
```

Es case-sensitive

```
//JS ES Case Sensitive, esto quiere decir que  
//var variable1 es diferente de var Variable1  
var variable1 = 1;  
var Variable = 1;  
// variable1 ≠ Variable
```

No es necesario terminar cada sentencia con el carácter de punto y coma.

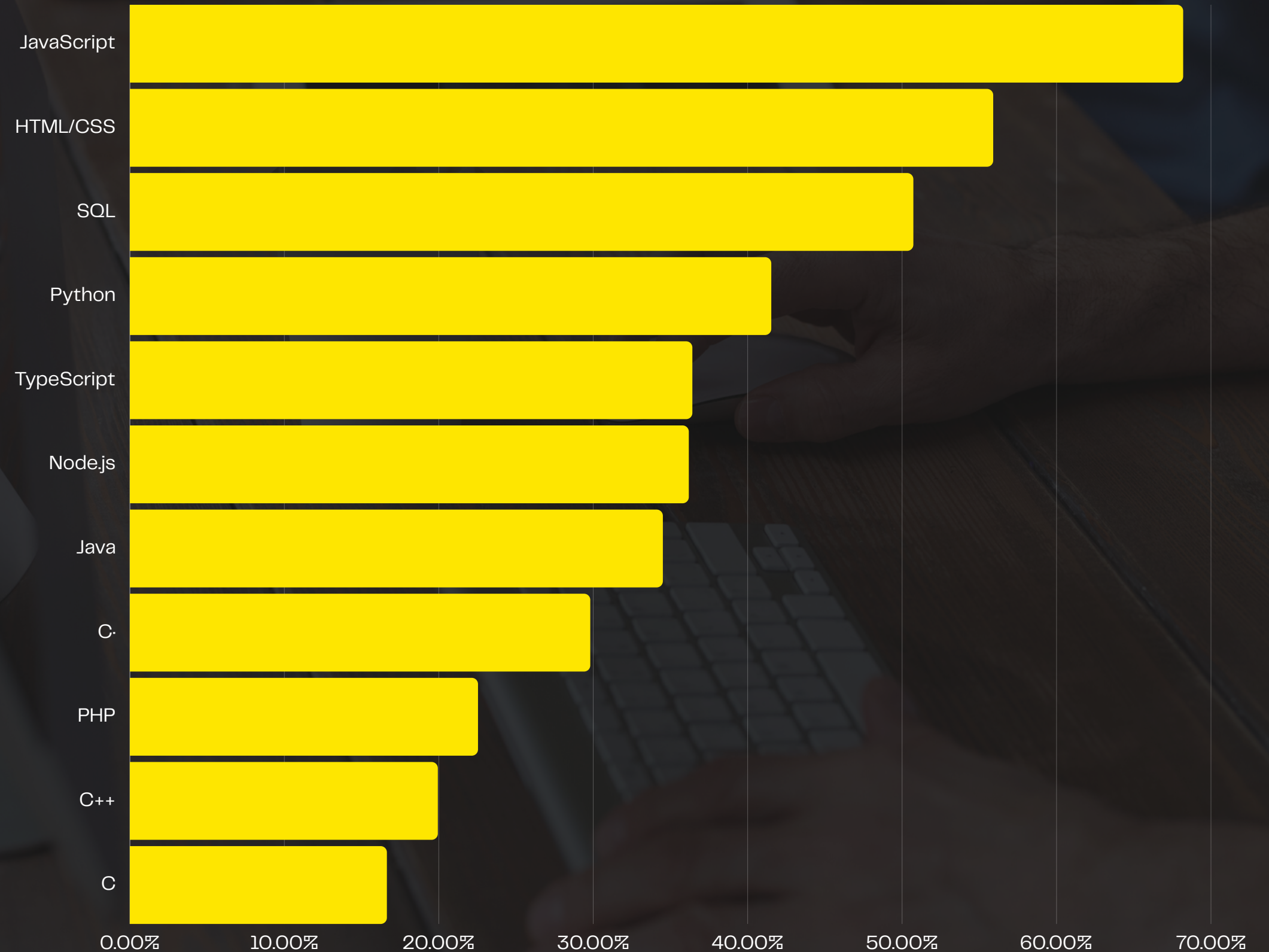
```
//En JS, no es necesario terminar cada sentencia  
//con el caracter de punto y coma  
let nombre = "Carlos"  
let edad = 21;  
let estudiante = true  
  
console.log(nombre, edad, estudiante);
```

Se puede incluir comentarios

```
//En JS, Se pueden incluir comentarios  
let nombre = "Carlos"  
// Comentario de una linea  
let edad = 21;  
/*  
    Comentario  
    de  
    n  
    lineas  
*/  
let estudiante = true
```

Lenguajes de programación más populares 2024

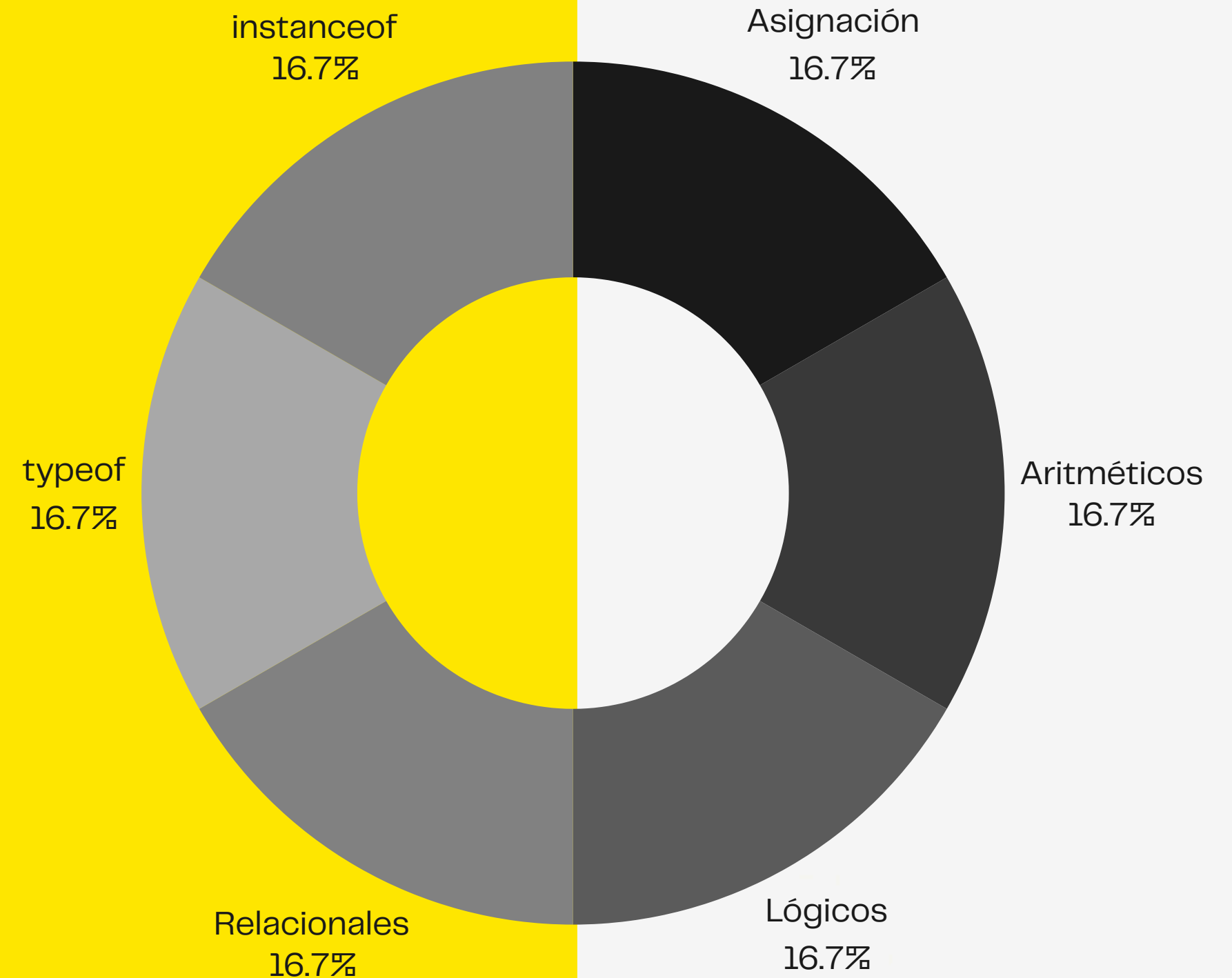
JS se mantiene como el lenguaje de programación más popular del mundo



Operadores

Permiten manipular el valor de las variables, realizar operaciones matemáticas con sus valores y comparar diferentes variables.

De esta forma, los operadores permiten a los programas realizar cálculos complejos y tomar decisiones lógicas en función de comparaciones y otros tipos de condiciones.



Tipos de operadores

PDC1

```
var numero1 = 3;
```

Asignación

Se utiliza para guardar un valor en específico

```
resultado = numero1 / numero2;  
resultado = 3 + numero1;  
resultado = numero2 - 4;  
resultado = numero1 * numero2;  
resultado = numero1 % numero2;
```

Aritméticos

Se utiliza para realizar manipulaciones matemáticas

```
expr1 && expr2
```

```
expr1 || expr2
```

```
!expr
```

Lógicos

Se utilizan para tomar decisiones sobre las instrucciones a ejecutar

Tipos de operadores

PDC1

```
resultado = numero1 >= numero2  
resultado = numero1 <= numero2  
resultado = numero1 == numero2  
resultado = numero1 != numero2
```

Relacionales

Se utilizan para relacionar variables de cualquier operación compleja

```
typeof myFuncion
```

Typeof

Se emplea para determinar el tipo de dato que almacena una variable

```
variable1 instanceof String
```

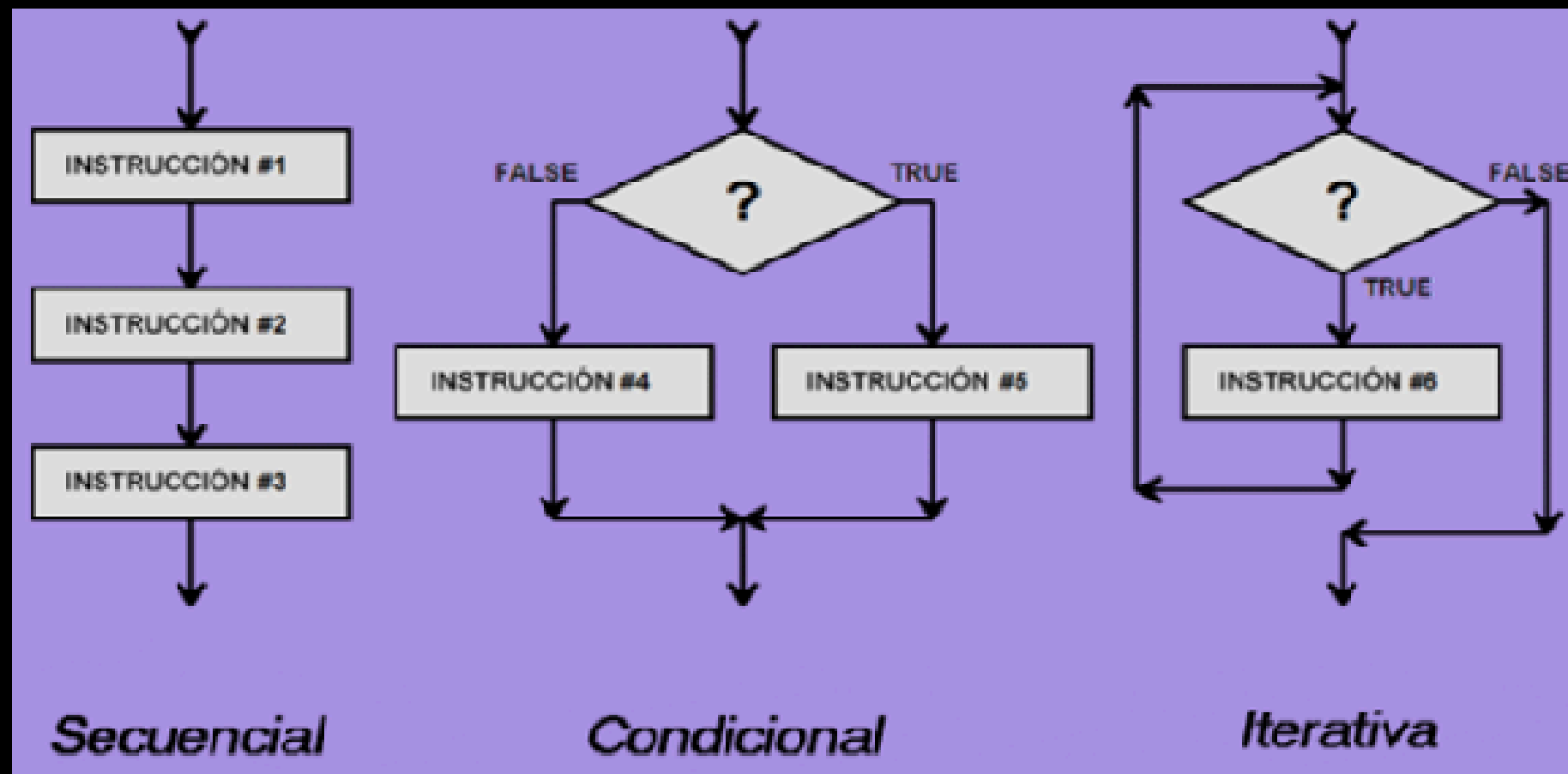
Instanceof

Se emplea para determinar la clase concreta de un objeto

Ejemplo de Operadores

```
main.scss sidebarComponent.svelte x ButtonComponent.svelte
src > components > sidebar > sidebarComponent.svelte >
You, 2 months ago | 1 author (You)
1 <script>
2   import sidebarController from './sidebarController.js'
3   import mementoes from '../store.js'
4   import './sidebar.scss'
5 </script>
6
7 <div class="sidebar">
8   <button class="add-memento" on:click={sidebarController.addMemento}>
9
10  <ul class="mementoes">
11    {#each $mementoes as memento}
12      <li class="memento-item" class:active={memento.active}>
13        on:click={() => sidebarController.selectMemento(memento.id)}
14        {memento.title}
15      </li>
16    {/each}
17  </ul>
18 </div>
```

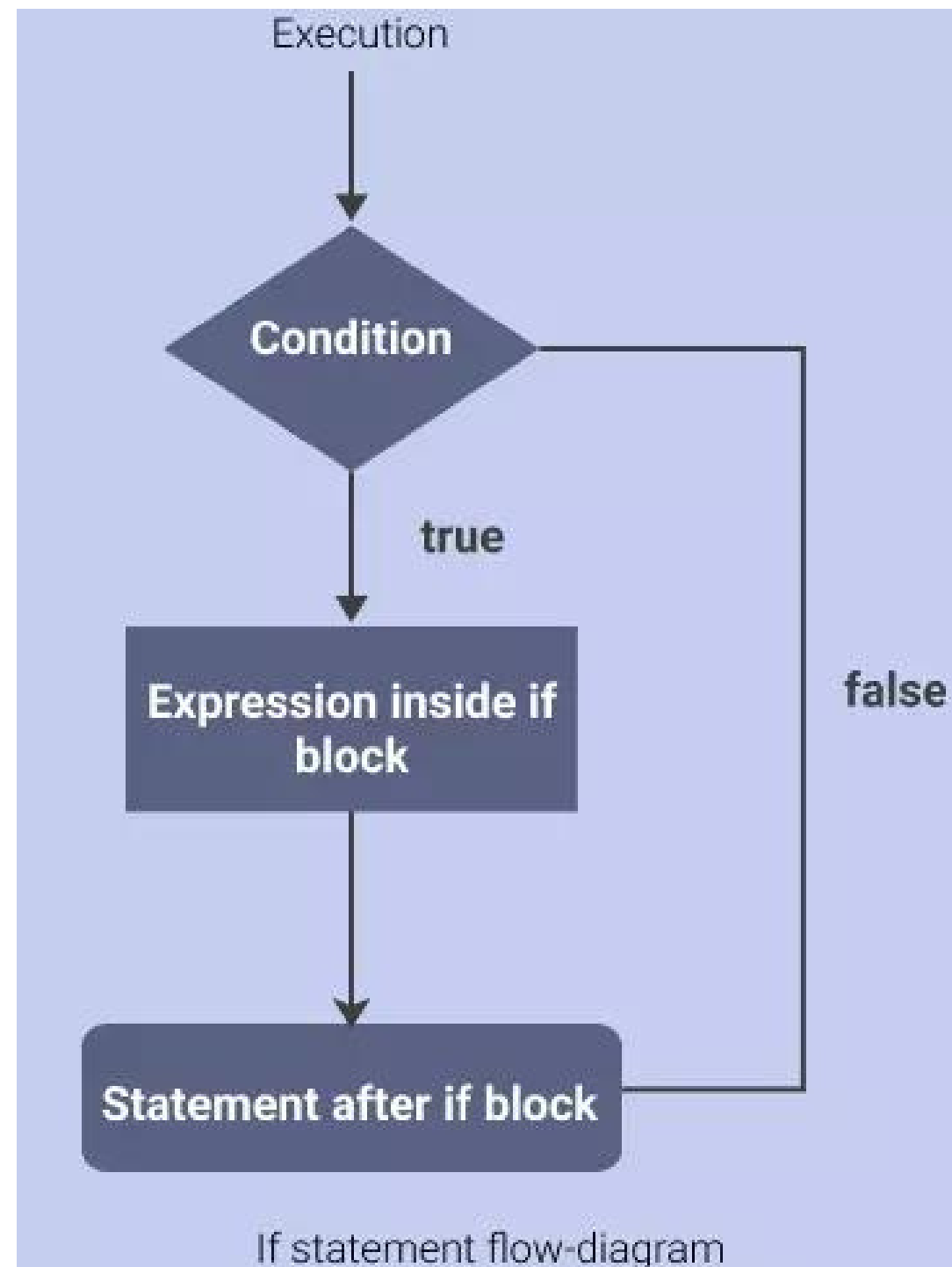
Estructuras de control en JS



Son instrucciones necesarias para realizar tareas distintas dependiendo del estado de nuestras variables o realizar un mismo proceso muchas veces sin escribir las mismas líneas de código una y otra vez.

Estructura de control IF...ELSE

PDC1

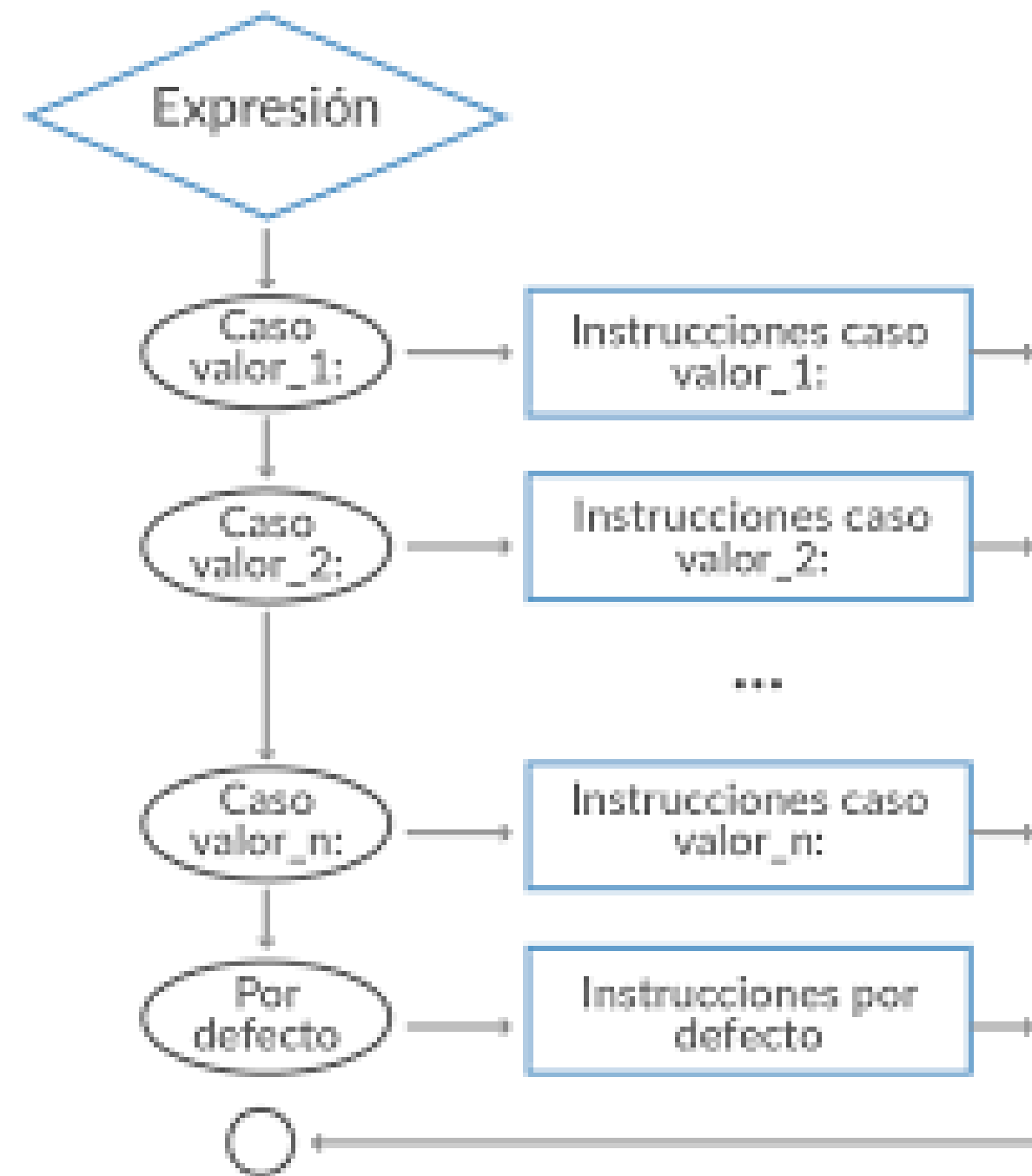


```
let condicion = true;

if(condicion) {
    console.log("Condicion verdadera");
}else {
    console.log("condicion falsa")
}
```

Estructura de control SWITCH....

PDC1



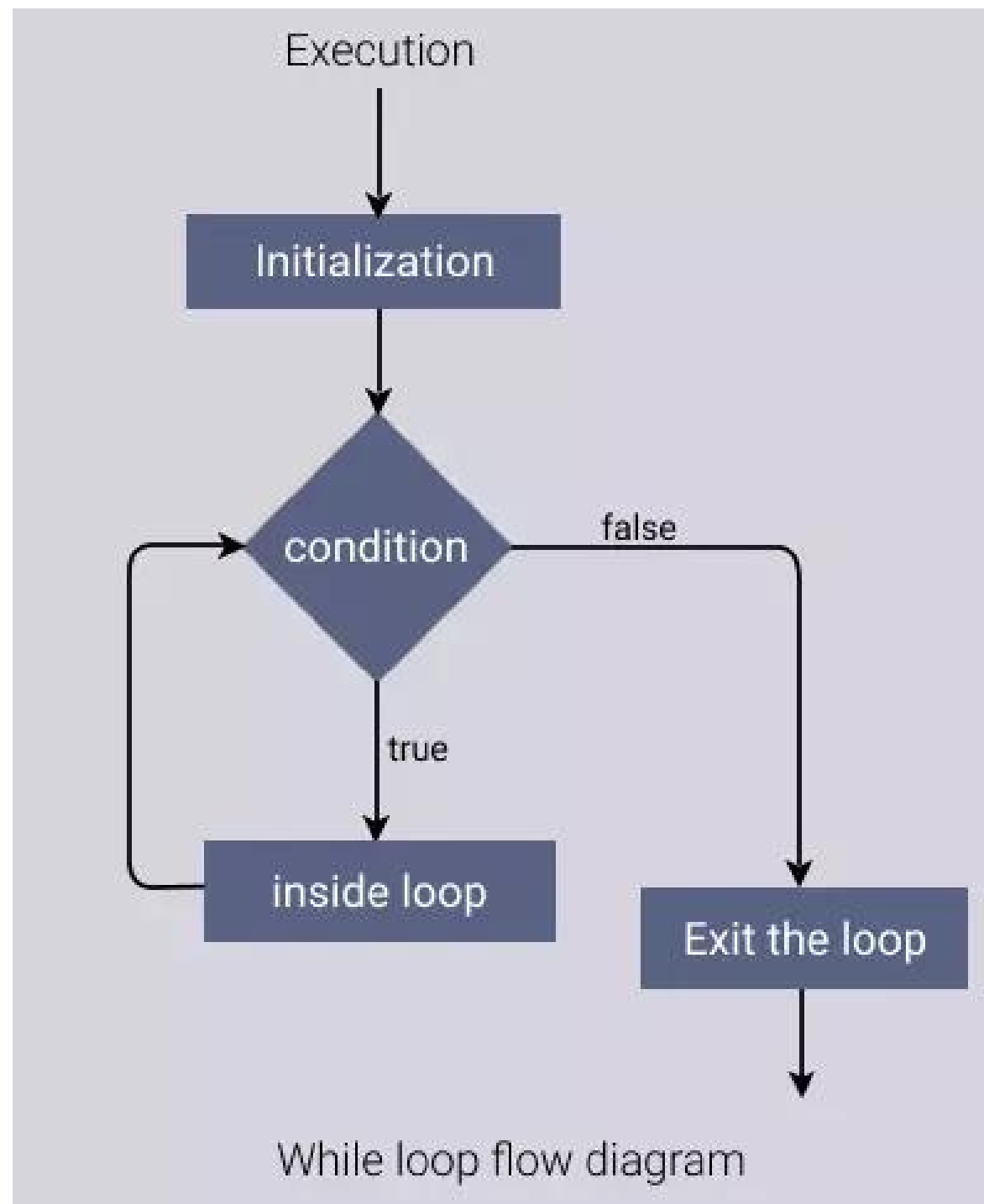
```
let condicion = 2;

switch (numero) {
  case 1:
    numeroTexto = "Numero uno";
    break;
  case 2:
    numeroTexto = "Numero dos";
    break;
  case 3:
    numeroTexto = "Numero tres";
    break;
  default:
    numeroTexto = "Numero no encontrado";
    break;
}

console.log(numeroTexto);
```

Estructura de control **WHILE....**

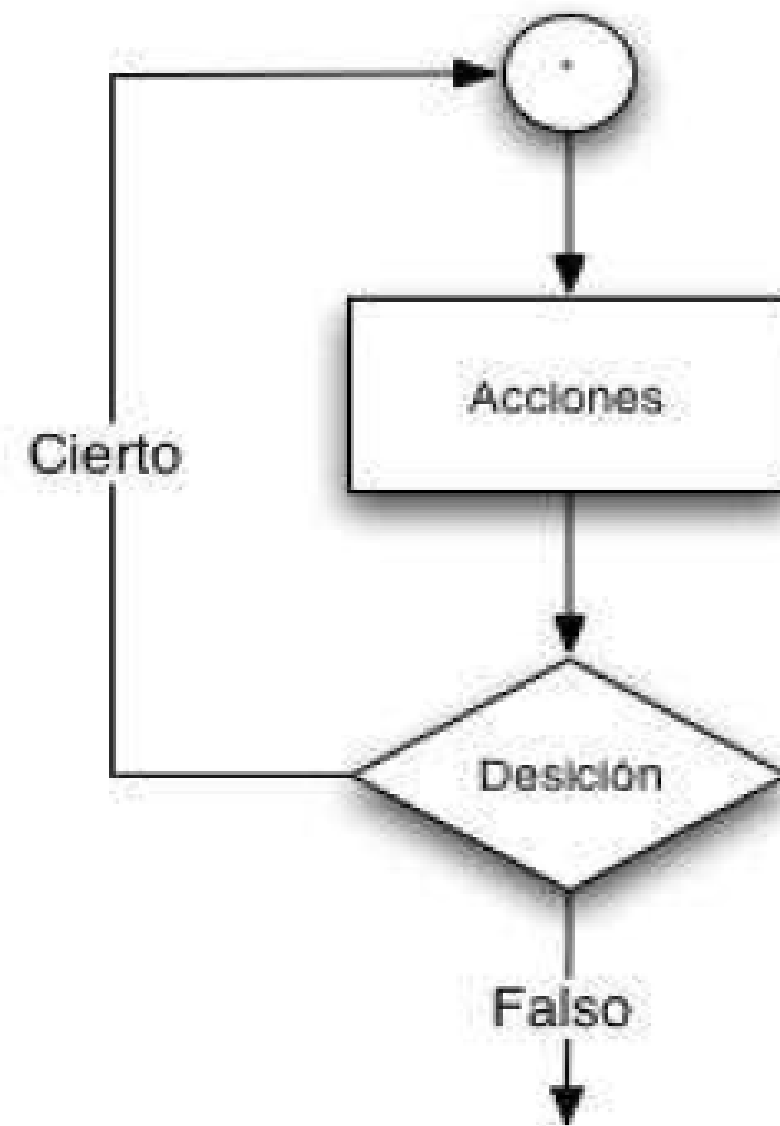
PDC1



```
let contador = 0;
while (contador < 10) {
    console.log(++contador); // 1,2,3,4,5,6,7,8,9,10
}
console.log("Fin ciclo while");
```

Estructura de control DO...WHILE....

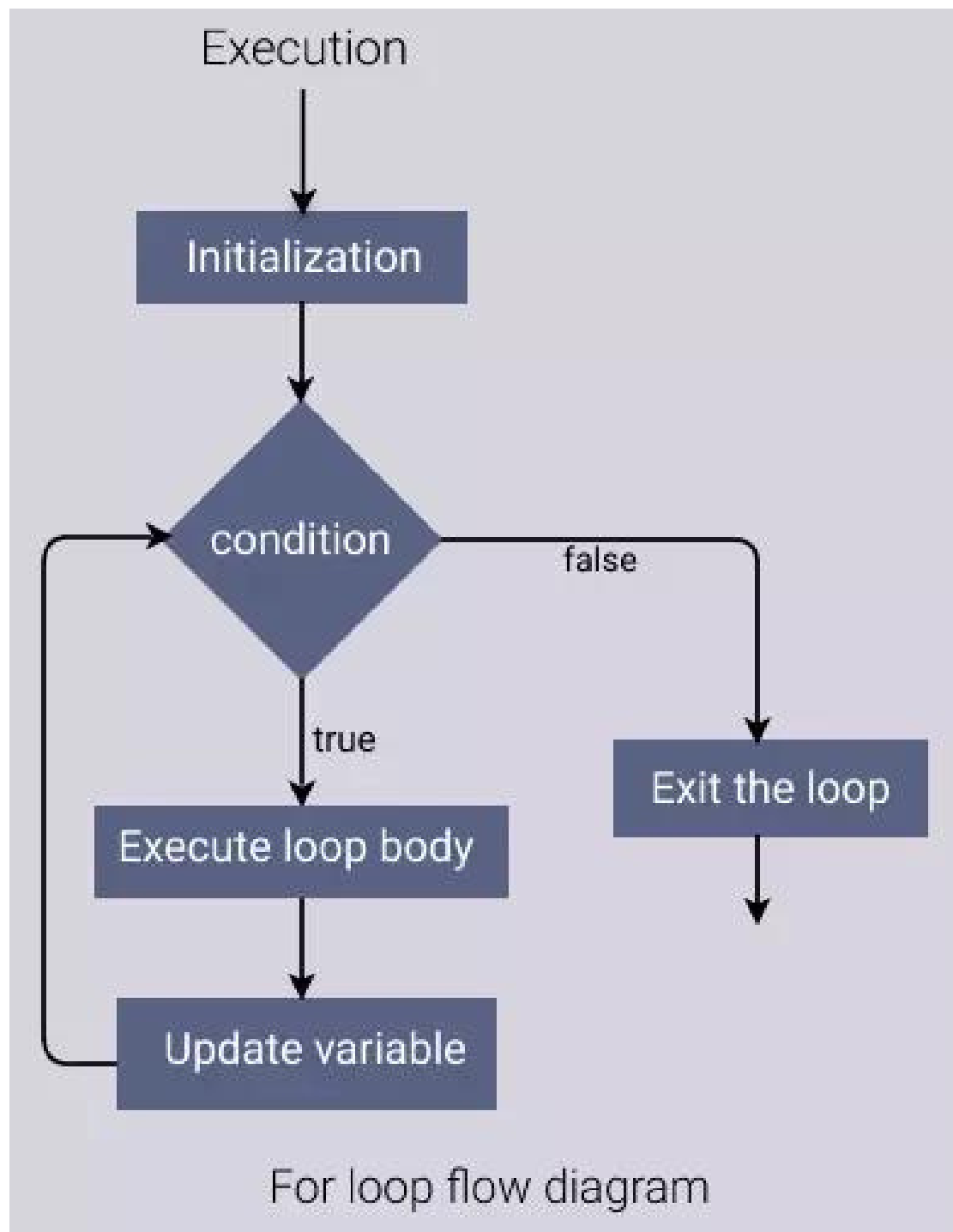
PDC1



```
let contador = 0;  
do {  
    console.log(contador++); //0,1,2,3,4,5,6,7,8,9.  
} while (contador < 10);
```

Estructura de control FOR....

PDC1



```
for (let contador = 0; contador < 10; contador++) {  
  if (contador % 2 === 0) {  
    console.log(contador);  
    break;  
  }  
}  
console.log("Fin ciclo for");
```

Funciones en JS

PDC1

¿Qué es una función?

Es una secuencia con nombre de declaraciones que realiza una operación específica.

¿Por qué usar un función?

Sencillamente para **dividir un programa** en piezas más pequeñas y reutilizables.

Construcción básica de una función

```
function nombreFuncion(param1, param2, ...) {  
  // declaraciones  
  return ;  
}
```

return

Las funciones no solo pueden recibir valores o datos, también pueden o no devolver los valores.

() => { }

Función de flecha en JavaScript

Es una de las características introducidas en ES6 para una mejor escritura en JS.

Función Flecha

() ⇒ { }

Es una alternativa compacta a una función tradicional



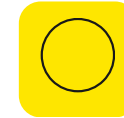
No tiene sus propios enlaces a this o super y no se debe usar como método.



No tiene argumentos o palabras clave.



No apta para los métodos call, apply y bind, que normalmente se basan en establecer un ámbito o alcance.



No se puede utilizar como constructor.



No se puede utilizar yield dentro de su cuerpo.

**¿Tienes alguna
pregunta?**



Espero, que hayas aprendido algo nuevo.



**Gracias por su
atención**

**Construye algo que
el encante a 100
personas, no solo
algo que le guste a
1000 personas.**

Brian Chesky

